

LLM Architecture and Planning Systems



Designing and Deploying Intelligent Agents



Engage and Think



Imagine you are a Product Manager handling multiple product launches. Every day, your inbox is flooded with customer queries, your team looks to you for quick answers, and you're expected to make fast decisions with limited information. It feels overwhelming, and you wish there was a smarter way to handle it all.

Now think about this: If you had a system that answered questions, gave quick insights, and made teamwork smoother, how much easier would your job be? Would learning how to build such systems be a game-changer for you as a Product Manager?

Learning Objectives

By the end of this lesson, you will be able to:

- 🔗 Implement planning techniques in AI agents to guide decisions in a way that feels deliberate and intelligent
- 🔗 Apply tools and memory components to give agents the ability to recall and respond more naturally
- 🔗 Build retrieval pipelines to power agent answers with knowledge that feels accurate and trustworthy
- 🔗 Create multi-agent workflows to enable teams of agents to collaborate smoothly, much like people working together
- 🔗 Design agent-to-agent communication systems to ensure seamless interactions between agents, just like humans

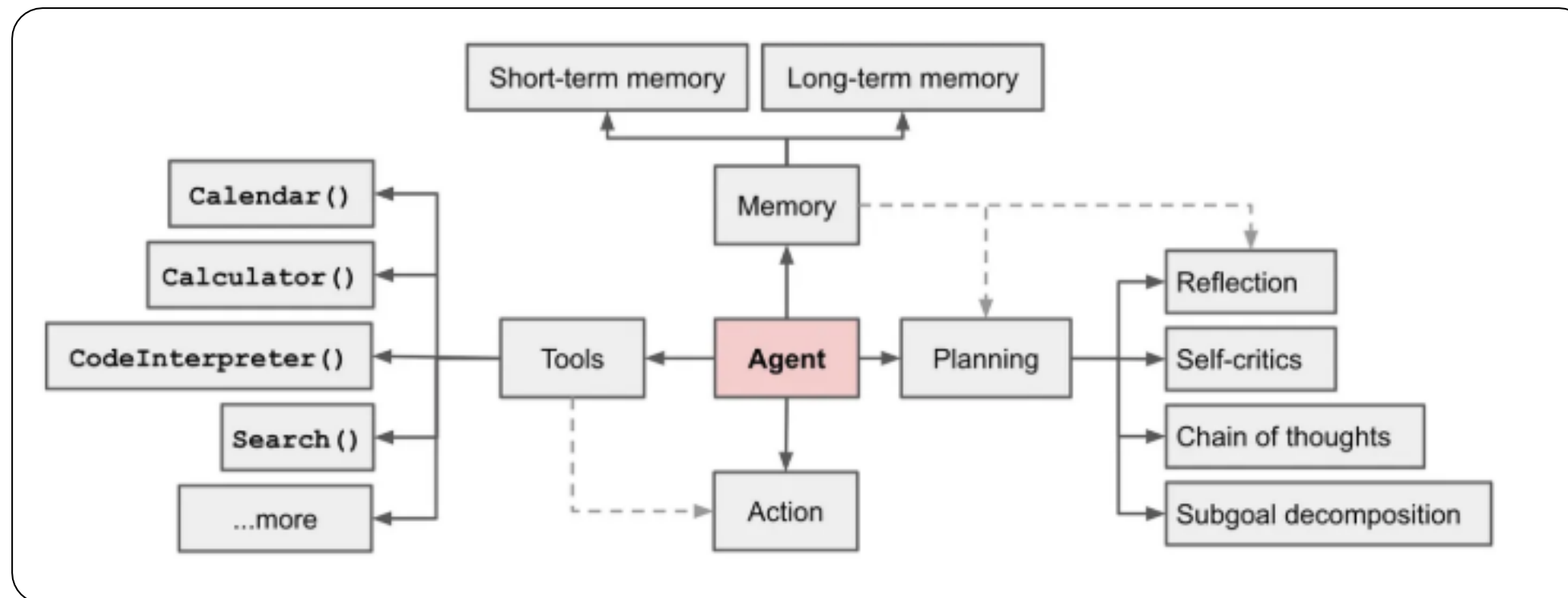




Planning for LLM Agents

How Planning Works in the LLM Agent Framework

Planning involves generating a sequence of actions, often using tools and memory, to solve multi-step or goal-oriented tasks. The following diagram shows this process in action:

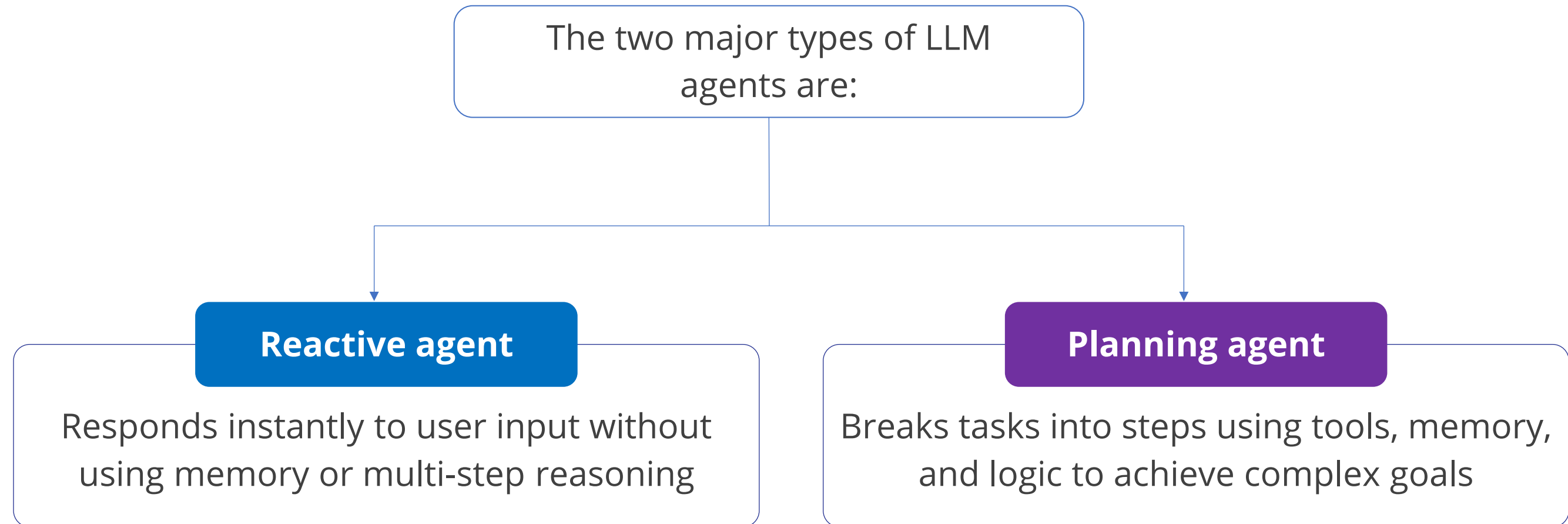


It is a crucial component that allows LLM agents to go beyond simple, one-off responses and engage in more sophisticated, multi-step problem-solving.

Before discussing *planning* in detail, let us first look at the types of LLM agents and how they can be designed to handle tasks.

Types of LLM Agents

They can be broadly categorized based on how they handle tasks and reasoning steps.



Reactive Agent vs. Planning Agent

LLM agents can be designed to handle tasks either reactively or through detailed planning, depending on task complexity and goals.

Feature	Reactive agents	Planning agents
Decision approach	Acts on current input instantly	Considers consequences before deciding
Memory usage	Does not store memory or track state	Maintains memory of past events and context
Response speed	Responds quickly in real time	Responds more slowly due to planning overhead
Example use cases	Handles FAQs, IoT tasks, and game character actions	Manages travel planning, virtual assistant tasks, and dashboard workflows

Benefits of Using Planning for LLM Agents

Planning offers the following advantages to help LLM agents solve complex tasks more effectively:



When Should PMs Use Planning?

Planning helps LLM agents behave more intentionally, reduce errors, and build trust.

Use planning when:

- Handling tasks with multiple steps
- Combining actions across different tools
- Adapting responses based on user input

Example use cases:

- Competitor analysis agent
- Day planner assistant
- Data summary chatbot



Effective planning often begins with well-structured prompts that guide agents' behavior and decision-making.

How Prompting Drives Planning Within LLM Agent Workflows

Prompts define what the user wants the agent to accomplish.

Prompts specify how the agent should think, reason, or act.

Prompts trigger multi-step actions like tool use and decision-making.

Prompts break complex goals into step-by-step instructions.

Prompting vs. Planning

While prompting can initiate planning, they operate at different levels of complexity.
Here are their differences:

Prompting

- Elicits one response
- Follows one-turn logic
- Handles short, simple tasks
- Example: Summarize this email.

Planning

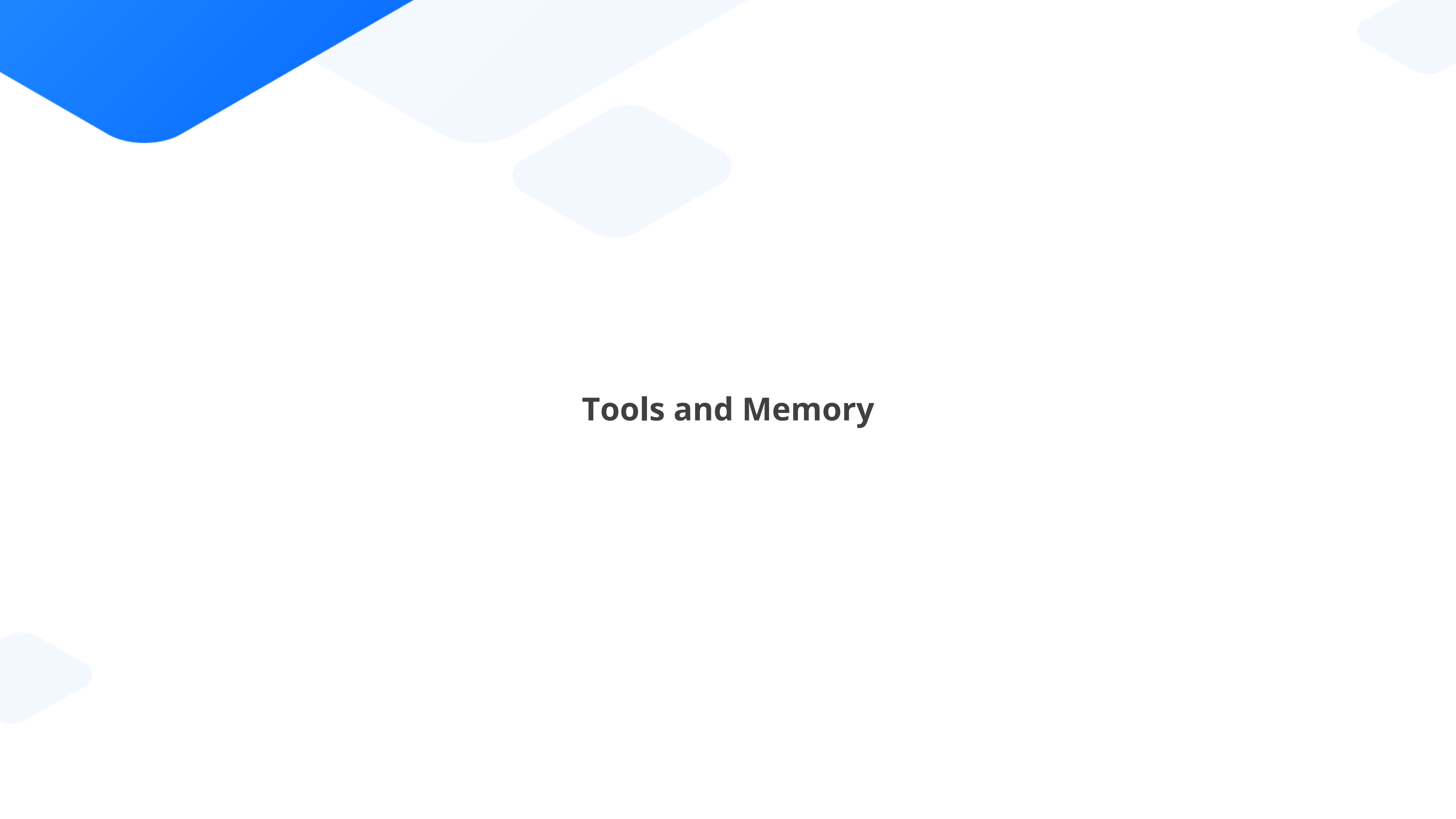
- Manages a task sequence
- Executes multi-turn steps
- Supports tool-driven flows
- Example: Check emails, summarize them, and send relevant reminders.

Quick Check



What role do prompts play in enabling planning in LLM agents?

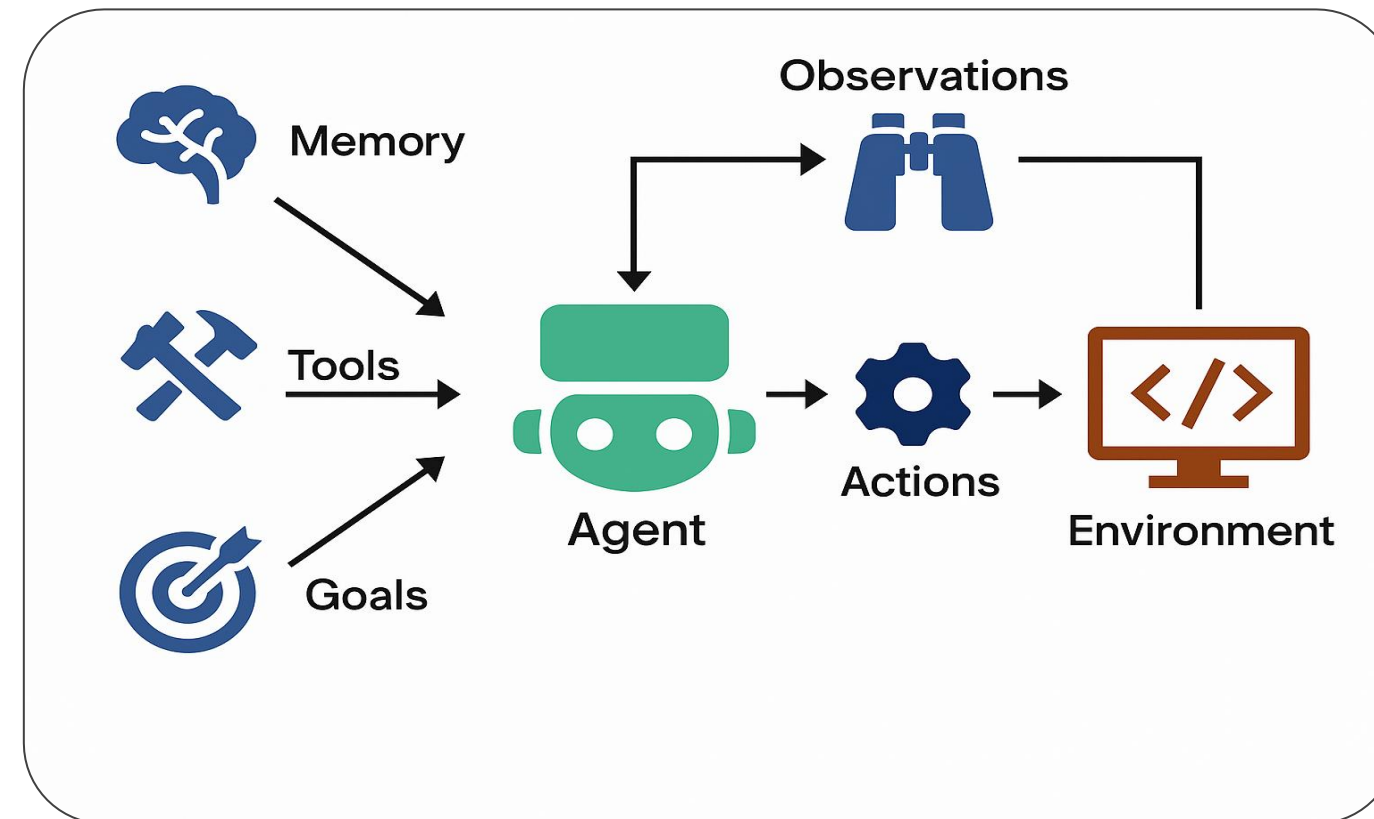
- A. They generate final outputs directly.
- B. They simulate memory functions.
- C. They instruct the agent to recall training data only.
- D. They guide multi-step reasoning and tool use.



Tools and Memory

Understanding Tool Interfaces

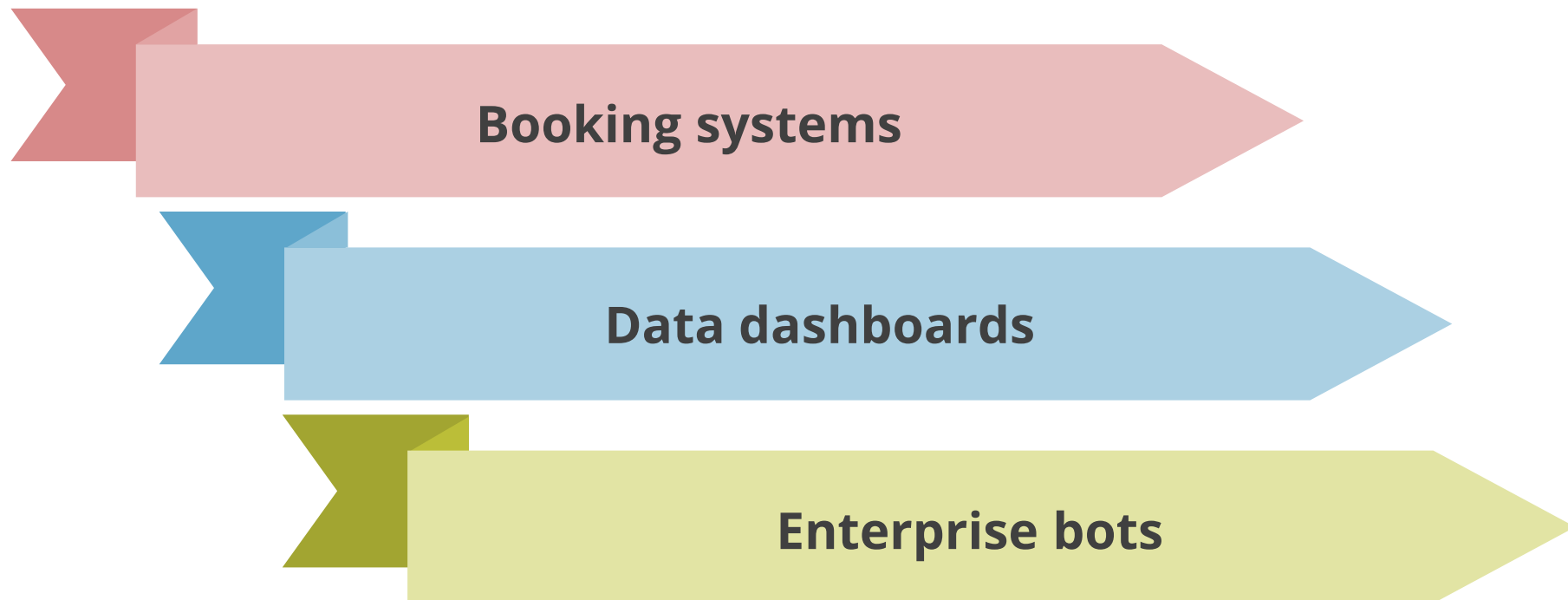
Tool interfaces enable an LLM to interact with the external world beyond its training data.



The agent uses **Tools** to perform actions within the environment and receives **Observations**, such as API responses, search results, or calculation outputs, to decide what to do next.

Why Agents Use Tools

Tools enable agents to perform real-time lookups, calculations, or actions, helping them answer questions they don't know natively. They are critical for use cases like:



Booking systems

Data dashboards

Enterprise bots

Tool Components

Each tool has the following:

Name

Define what the tool does

Function description

Describe what it needs
as input

Input and output field

Specify the expected input
format and the output type



These components enable seamless integration of external tools into LLM workflows, extending the agent's real-world capabilities.

Example: LLM Agent Using a Weather Tool

Let's look at how an LLM agent thinks, acts, and responds using a weather tool.

User

Will it rain in Delhi tomorrow?

LLM

Thought: I need to check the forecast.

Action

Call **get_weather(city=Delhi, date=tomorrow)**

Observation

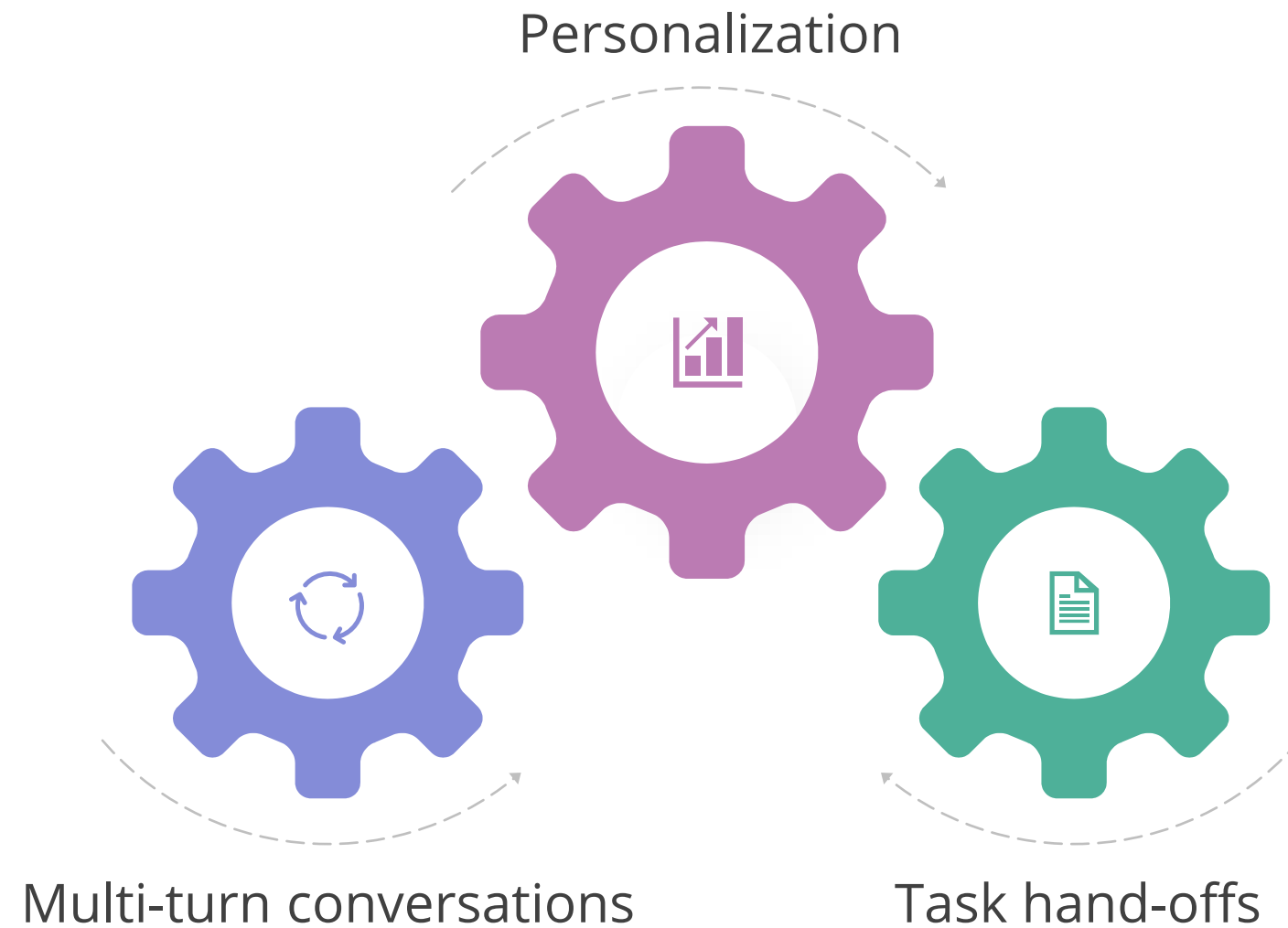
Rain is expected; 70% chance

Final answer

Yes, there's a 70% chance of rain in Delhi tomorrow.

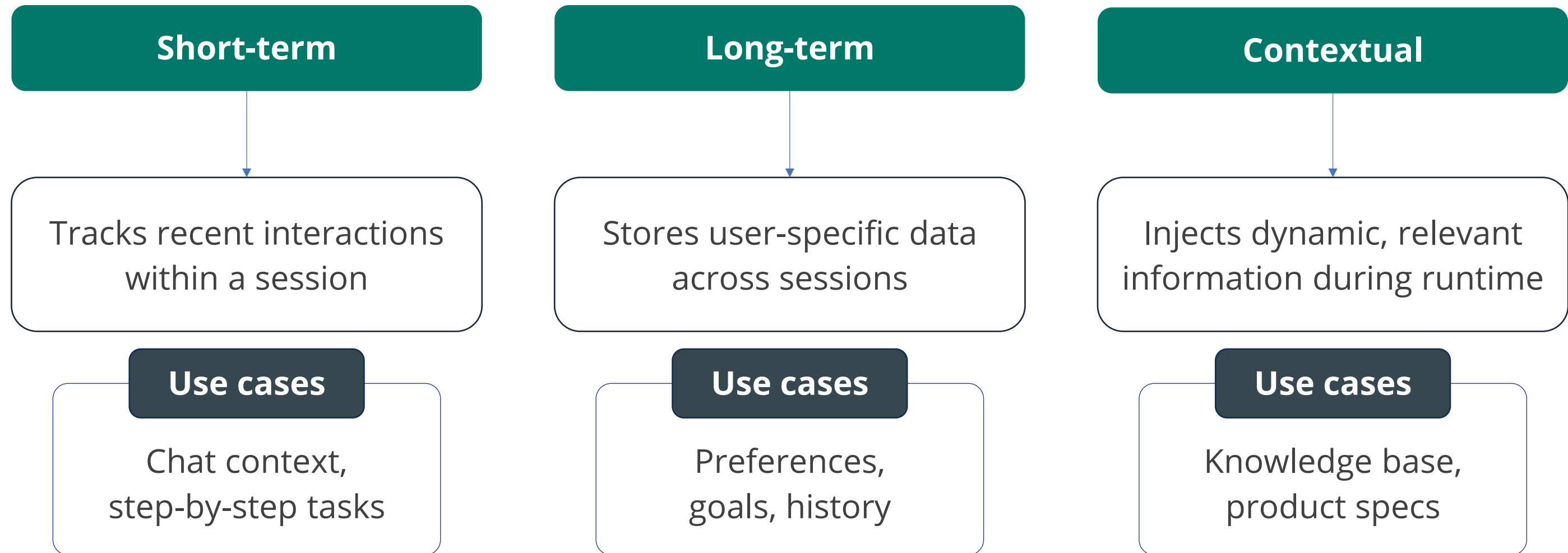
Why Agents Use Memory

Memory enables agents to track conversations, remember goals, and adapt. Without memory, each interaction is treated like a blank slate. It is useful for tasks like:



Memory Types

There are three key types of memory that LLM agents use:



How Memory Functions for LLM Agents

LLM agents rely on structured memory systems to operate intelligently. These systems must be configured, updated, and managed carefully, as follows:

Configuring the memory system

Set what to store, when to update, and where to save it

Controlling retention and forgetting

Decide what to keep or discard using time or size limits

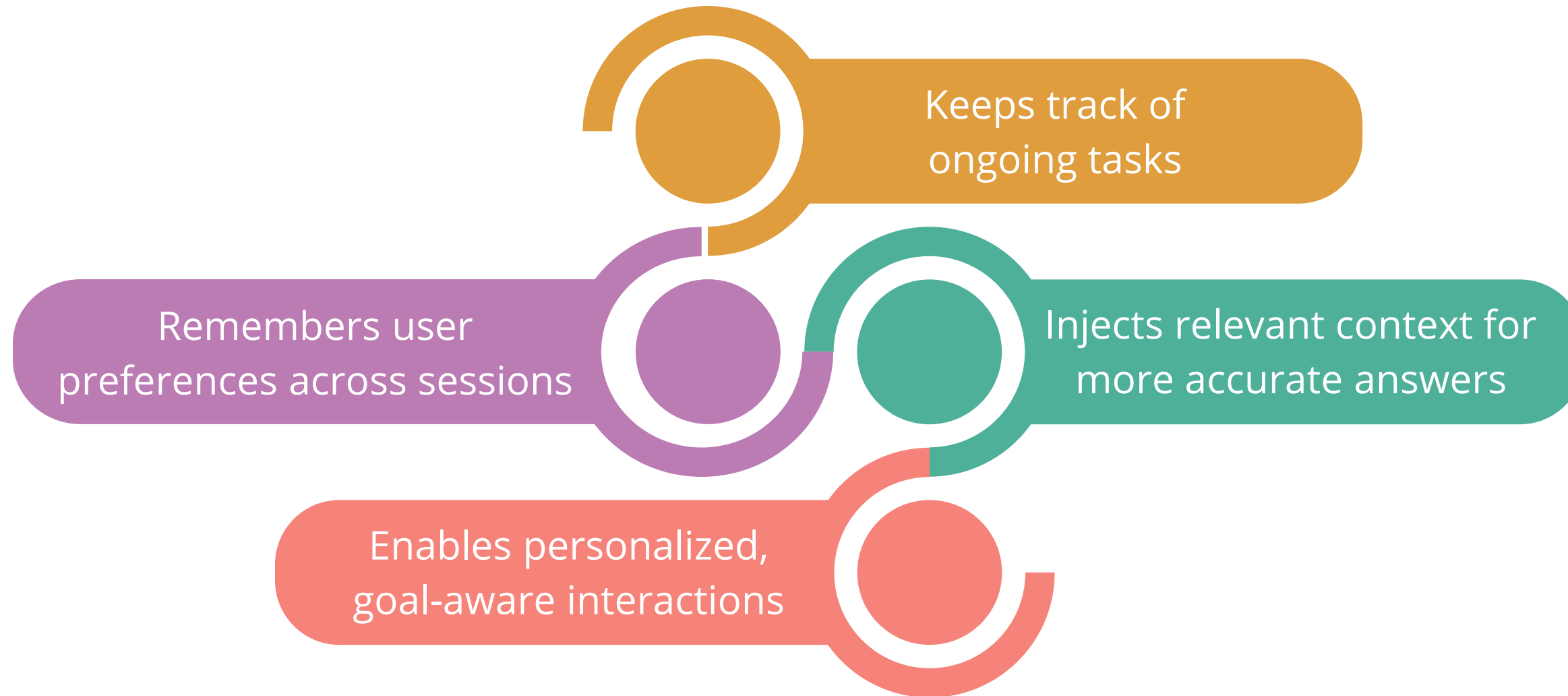
Determining relevant information

Filter memory based on task, similarity, or keywords

Preventing memory overload

Summarize, prune, or offload unused data smartly

How Memory Improves Agent Behavior

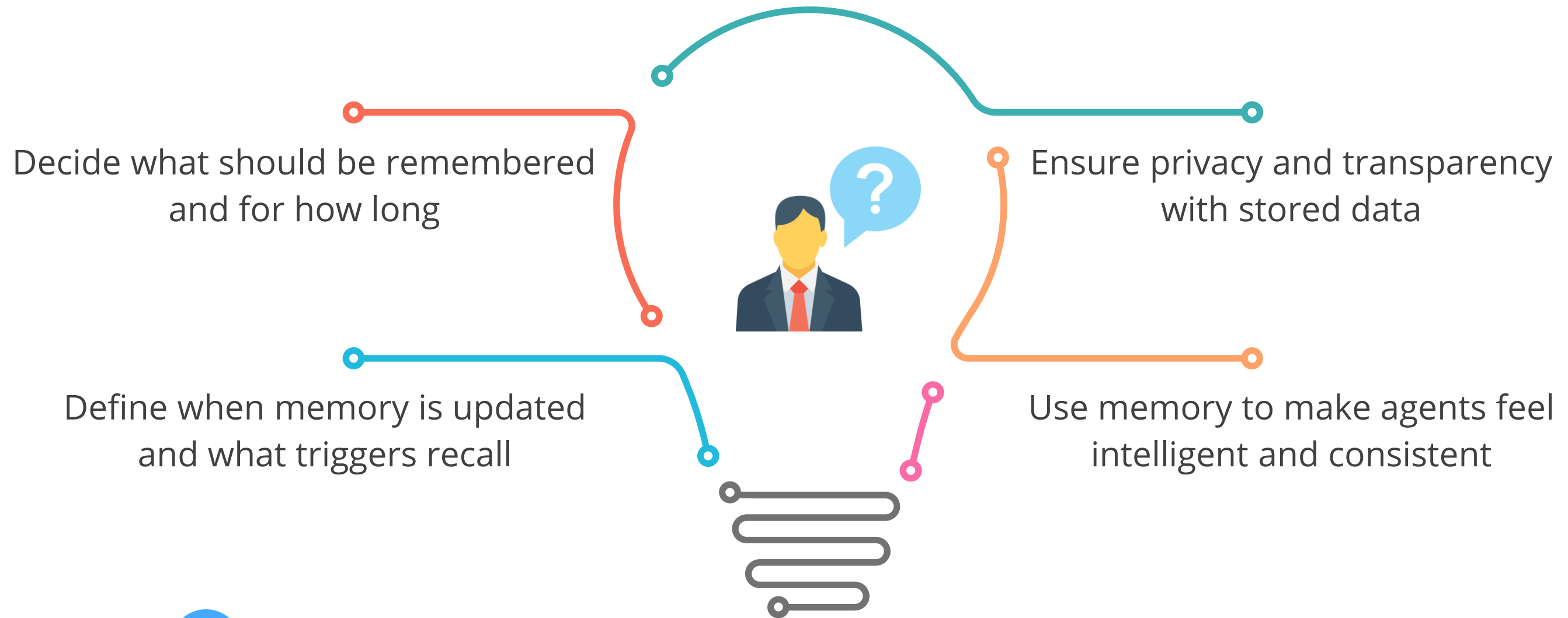


Note:

Memory allows agents to stay consistent, relevant, and user-aware throughout interactions.

Designing Memory: What PMs Need to Know

When designing memory for LLM agents, PMs should focus on these key considerations:



Thoughtful memory design enables smarter, more human-like agents.

Privacy and Consent in Agent Memory Design

The following are key considerations for storing and using user data in LLM agents:

Protecting user privacy

Secure data with encryption, access control, and legal compliance

Managing user consent

Get clear consent and let users opt in or out of data storage

Clarifying data purpose

Explain if data is used for training or improvement, and get consent

Balancing UX and safety

Use memory for better UX, but give users control over their data

Quick Check



What is a key benefit of long-term memory in agent systems?

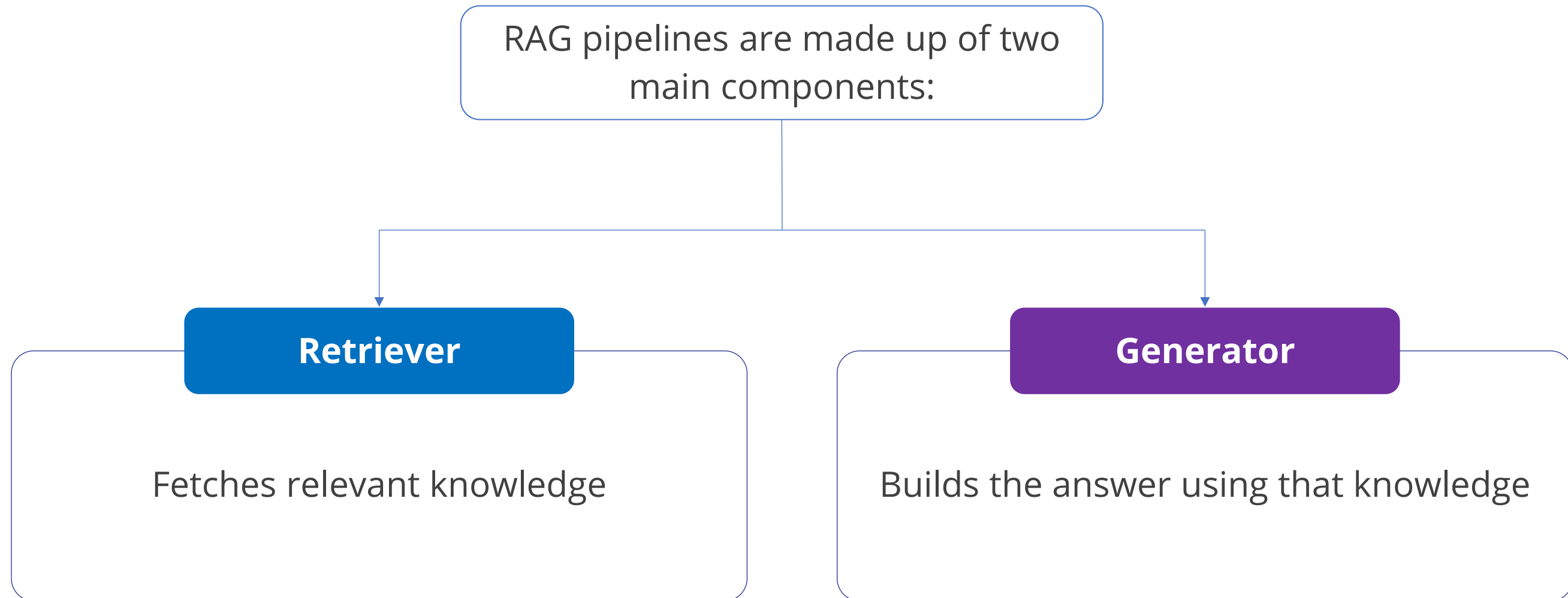
- A. The system enables faster response generation.
- B. It supports stateless interactions across sessions.
- C. It allows for the storage of past knowledge for future retrieval.
- D. It eliminates the need for context windows.



Retrieval-Augmented Generation (RAG)

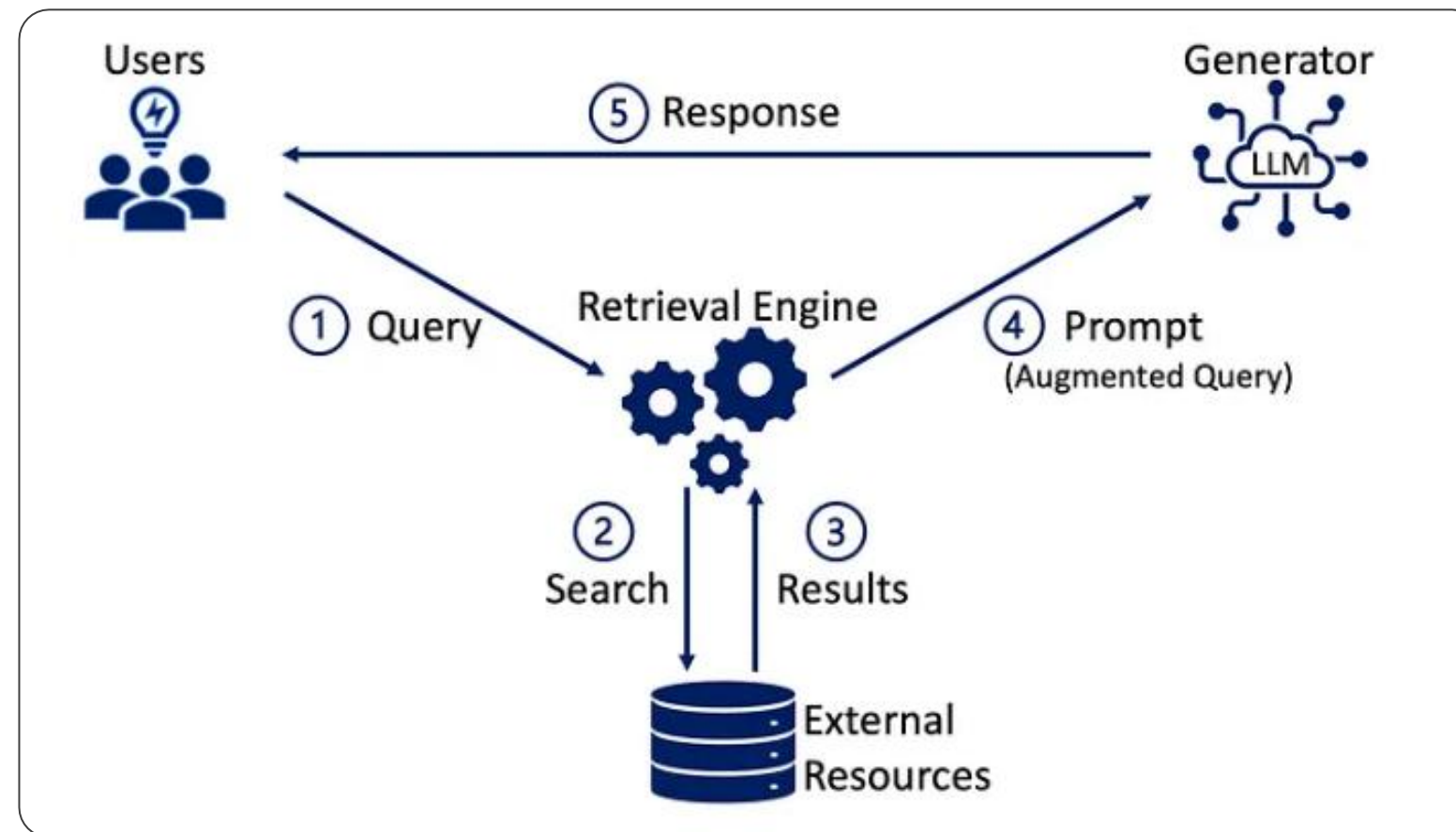
What Is Retrieval-Augmented Generation (RAG)?

It is a technique where an LLM pulls contextually relevant data from a knowledge source, such as documents or databases, and uses it to generate more accurate answers.



Overview of RAG Architecture

In practice, RAG starts with a user query, retrieves supporting information, and then composes a response using that evidence, all in real time.



Think of the retriever as a journalist collecting facts, and the generator as the editor shaping the article.

Query vs. Prompt in RAG Systems

In RAG, queries and prompts serve different roles. Understanding both is key to building accurate and grounded outputs.

Aspect	Query	Prompt
Definition	A search request sent to retrieve relevant documents.	The final input given to the language model.
Target	Retrieval engine or vector store	Language model (LLM)
Purpose	To fetch facts or context from external sources	To generate a final response using the query and retrieved content
Example	What are the symptoms of flu?	Using the information retrieved, explain flu symptoms to a 5th grader

The Retriever: Context Provider



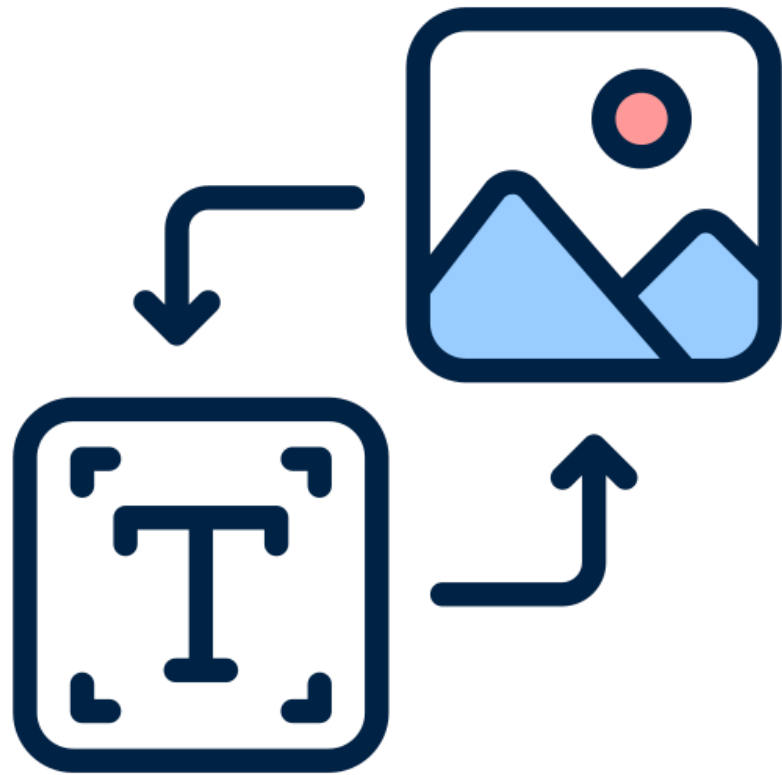
What it does:

- Converts the user's question into a search query
- Uses similarity search to find the most relevant text chunks
- Pulls information from a vector database (Example: FAISS, Pinecone)



The retriever fetches relevant context to guide accurate LLM responses.

The Generator: Answer Composer



What it does:

- Receives retrieved documents and the original query
- Synthesizes an answer using LLM reasoning
- Grounds the output in the source material
- Uses LLMs to generate fluent responses (Example: GPT-4, Claude)



The generator retrieves context and leverages LLM intelligence to craft grounded, fluent answers.

Frameworks for Implementing RAG

Popular frameworks help developers implement RAG with modularity, scalability, and ease of integration. Some of the most widely used frameworks are:

LangChain RAG

The platform builds RAG pipelines using modular LLMs, retrievers, memory, and prompt chains.

Haystack

It offers open-source pipelines with retrievers, generators, and scalable orchestration.

OpenAI Assistants API

The system handles retrieval, function calls, and grounding via a unified OpenAI interface.

RAG: Real-World Examples

RAG is already being utilized across various industries to address real-world problems. Here are some practical examples of how it is making a difference:

Use case	How RAG helps
Customer support chatbot	Pulls the latest policies from internal documents
Sales assistant	Retrieves product specs from PDFs
HR assistant	Uses policy documents for accurate replies

Challenges with RAG

RAG systems face the following three key implementation challenges:

Latency

Slows down response time due to retrieval and external calls, especially with large documents or remote stores

Relevance

Fails to match user intent, reducing the usefulness of the generated output

Grounding

Strays from retrieved data, requiring alignment to ensure factual accuracy

Mitigating Challenges in RAG Systems

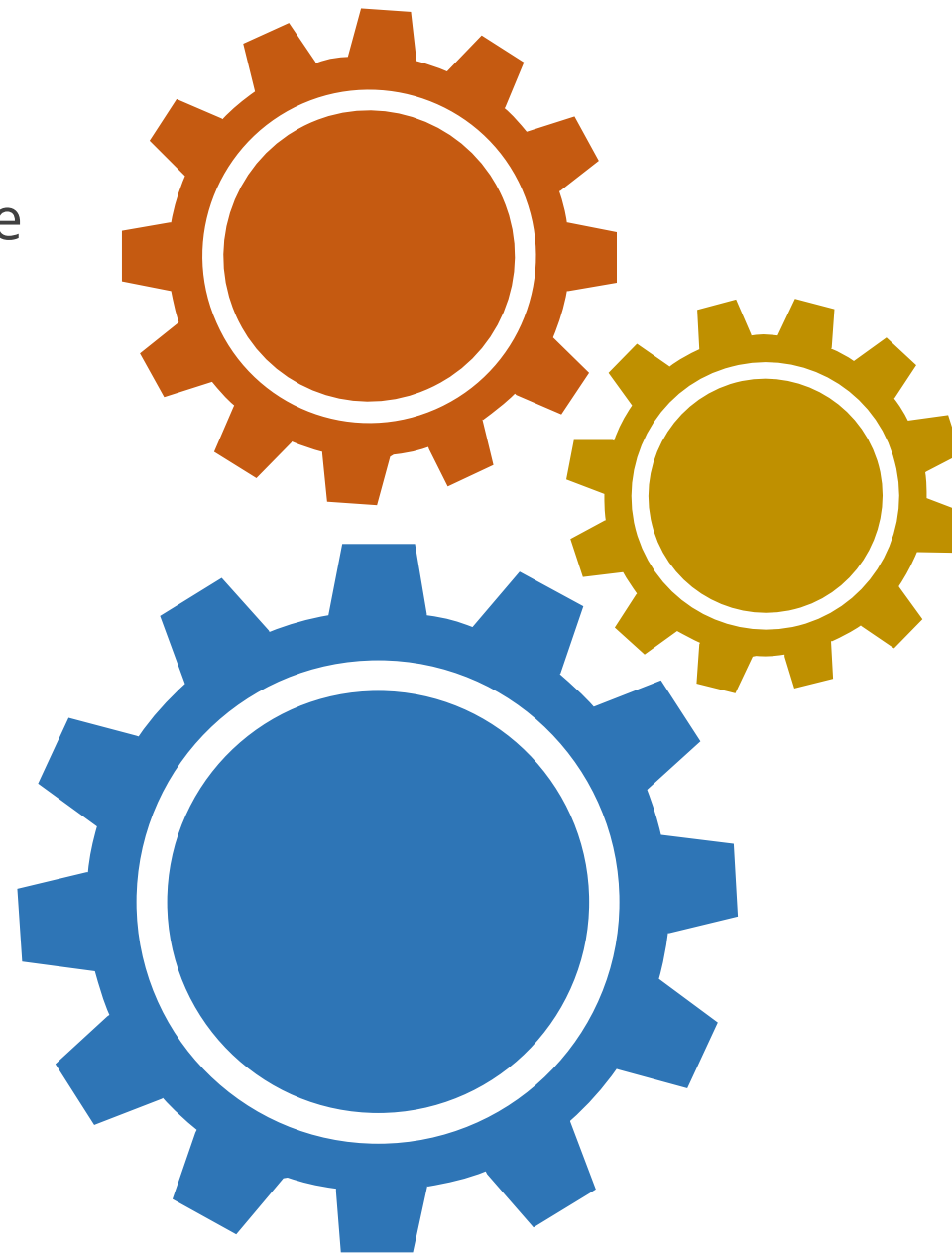
Simple strategies to handle common RAG issues:

Latency

Cache frequent queries and use smaller, faster data chunks

Grounding

Inject sources into prompts and validate model outputs



Relevance

Rewrite queries and apply filters to improve result quality

Quick Check



In a typical RAG pipeline, what is the function of the retriever component?

- A. Fine-tuning the model on task-specific data
- B. Scoring user prompts for quality
- C. Fetching relevant documents or chunks from a knowledge base
- D. Generating natural language responses

Demo: Building an RAG-Powered FAQ Agent with Custom Knowledge



Duration:15 minutes

Overview: This demo highlights how to build a Retrieval-Augmented Generation (RAG) agent capable of answering domain-specific FAQs. It highlights the process of retrieving relevant documents and generating grounded responses through a LangChain-powered pipeline.

Note:

Please download the demo document from the Reference Material section for step-by-step guidance.

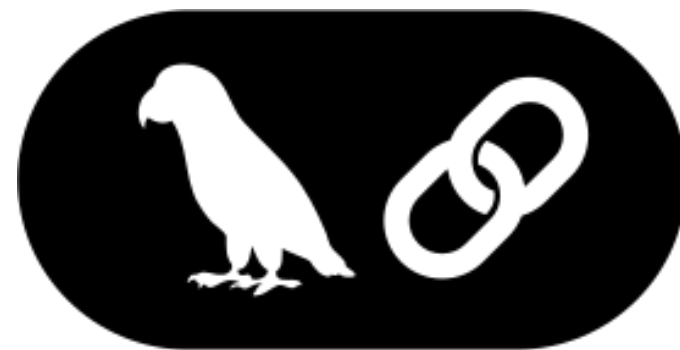
DEMONSTRATION



Introduction to LangChain Agents

What Is LangChain?

It is an open-source framework for building LLM-powered applications. It connects language models with external data, memory, and APIs to create dynamic, context-aware agents.



LangChain



LangChain enables PMs to design smarter AI workflows without starting from scratch.

Why LangChain Matters for PMs

Product Managers can benefit from LangChain in several ways. Its role includes:

Bridging AI and product workflows

Auto-tags feature requests from Slack to Trello using LangChain

Enabling faster prototyping

Builds an FAQ bot quickly using RAG template and public docs

Minimizing coding requirements

Deploys an HR onboarding assistant with memory and API access

Scaling agent-based solutions

Deploys a return-handling agent using APIs and internal docs

Types of Agents in LangChain

LangChain supports three agent types, each suited for specific use cases. They are:

Reactive agents

- ConversationalAgent
- ZeroShotAgent

Tool-using agents

- ReActChain
- SelfAskWithSearchChain

Planner-based agents

- MRKLChain
- StructuredChatAgent



Understanding these differences helps PMs select the appropriate agent type based on the task's complexity, required tools, and reasoning needs.

Reactive Agents

These are AI systems that respond immediately to user inputs without relying on memory, past interactions, or complex reasoning.

Capabilities

- Responds directly to user input without memory or history
- Requires no multi-step reasoning or external tool use
- Performs well in single-turn, factual interactions

Use case

FAQ bots that answer simple, single-turn questions like *What's your refund policy?*

Tool-Using Agents

These are AI systems that extend their capabilities by invoking external tools to perform complex or real-time tasks.

Capabilities

- Selects and executes tools such as APIs or calculators
- Solves tasks that require more than language generation
- Chooses tools dynamically based on user intent

Use case

An AI assistant that uses a weather API and a calculator to answer questions like
Should I carry an umbrella? How long will it take me to walk 3 km?

Planner-Based Agents

These are AI systems that tackle complex tasks by breaking them down into structured, sequential steps and managing execution across tools and memory.

Capabilities

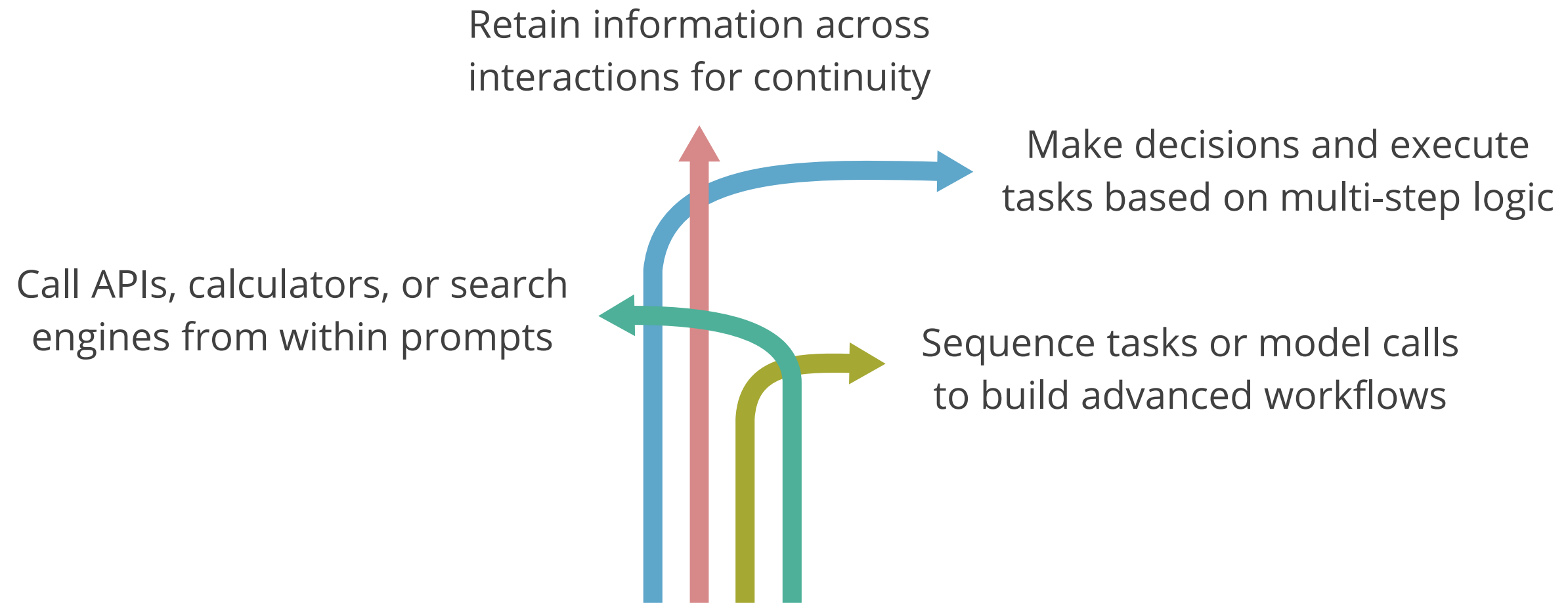
- Breaks high-level goals into smaller, manageable actions
- Combines planning, memory, and tool orchestration
- Handles multi-turn interactions and logical workflows

Use case

An AI travel planner that books flights, hotels, and suggests activities in sequence based on user preferences

Key Capabilities of LangChain

LangChain helps agents interact, reason, and operate effectively. It can:



To understand these capabilities better, let us look at how LangChain uses **tools** and **memory**, the building blocks of agent behavior.

What Are Tools and Memory in LangChain?

Tools and memory make LangChain agents more capable and intelligent.

Tools

Allow agents to interact with the external world, such as APIs, calculators, or databases

Memory

Allows agents to retain past inputs for context-aware, continuous conversations

To unlock these capabilities, agents must be equipped with properly configured tools and memory. Let us explore how to set them up in LangChain.

Step 1: Setting Up Tools

Load tools for your agent

1. Load utilities like web search, calculator, or APIs using **load_tools()**
2. Trigger tools when the agent needs to perform an action
3. Define custom tools for domain-specific tasks or requirements

```
from langchain.agents import load_tools

tools = load_tools(["serpapi", "llm-math"]) # Loads web
search and calculator tools
```

Step 2: Setting Up Memory

Add memory to retain context

1. Use built-in memory classes like **ConversationBufferMemory**
2. Store dialogue history between the user and the agent
3. Enable follow-up questions and personalized responses
4. Extend memory with vector stores for long-term use

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory(memory_key="chat_history")
```


Step 3: Attaching Tools and Memory to the Agent

Initialize the agent with tools and memory

1. Reason through tasks, use tools, and recall past exchanges
2. Handle multi-turn conversations and structured task flows

```
from langchain.agents import initialize_agent
from langchain.llms import OpenAI

llm = OpenAI(temperature=0)

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent="conversational-react-description",
    memory=memory,
    verbose=True
)
```

Quick Check



Why is LangChain particularly useful for Product Managers building AI features?

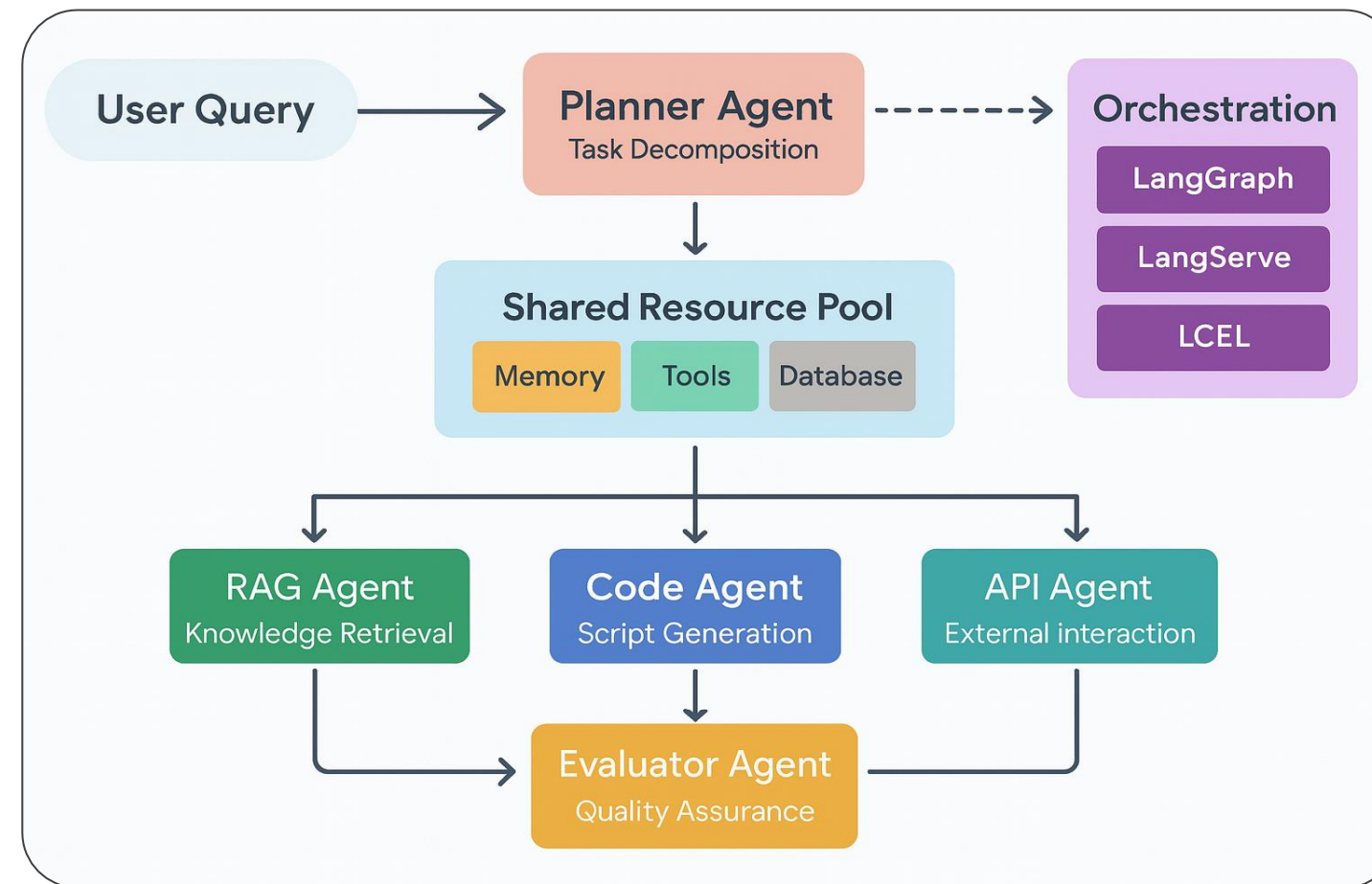
- A. It allows full model fine-tuning using low-code tools.
- B. It abstracts model training and removes the need for APIs.
- C. It provides modular tools to connect LLMs with memory, tools, and workflows.
- D. It replaces the need for prompt engineering entirely.



Introduction to the LangChain Agent Execution Loop

LangChain Agent Execution Loop

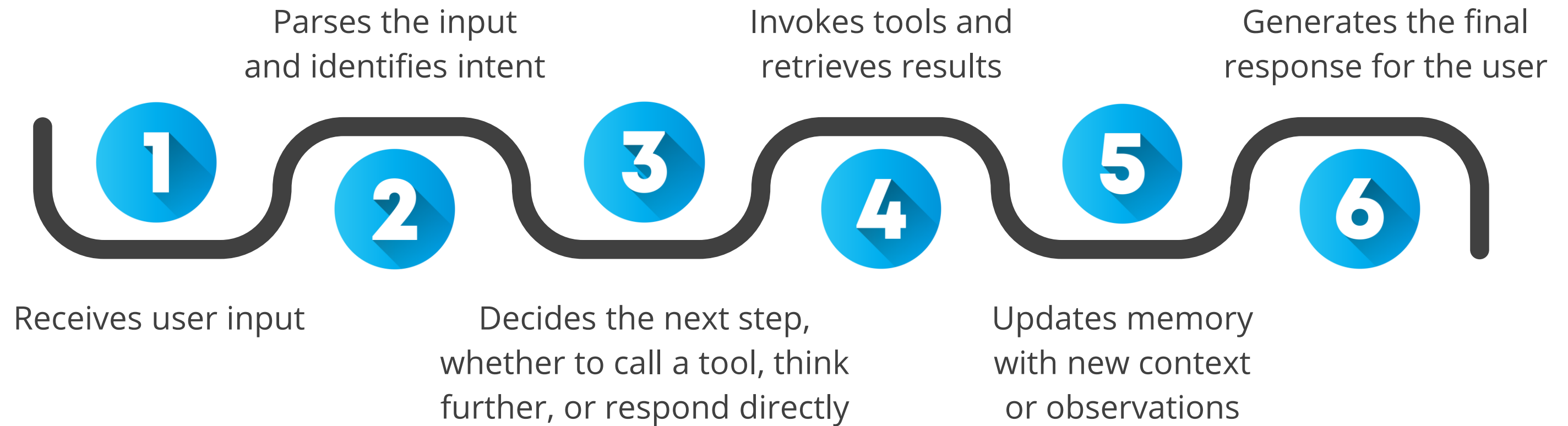
It guides agents to decide what to do next by coordinating planning, memory, tools, and logic in a structured, step-by-step process.



This loop transforms language models into decision-making systems that can solve complex tasks with precision, adaptability, and transparency.

Flow of Agent Execution

The following diagram shows how an LLM agent actively processes and responds to a user query through six steps:



This structured loop lets agents respond intelligently, just like a skilled assistant.

Code Example: Agent Execution Flow in LangChain

Step 1: Import the required libraries

```
from langchain.agents import load_tools, initialize_agent
from langchain.agents.agent_types import AgentType
from langchain.llms import OpenAI
```

- **load_tools, initialize_agent:** Functions to load utilities and set up the agent.
- **AgentType:** Defines different agent behaviors.
- **OpenAI:** Module for connecting to OpenAI's language models.

Code Example: Agent Execution Flow in LangChain

Step 2: Load tools

```
tools = load_tools(["serpapi", "llm-math"])
```

- **load_tools(["serpapi", "llm-math"])** equips the agent with web search and mathematical calculation capabilities.
- **serpapi:** This tool performs web search using SerpAPI.
- **llm-math:** This tool performs math operations like square root calculation.

Code Example: Agent Execution Flow in LangChain

Step 3: Initialize the LLM

```
llm = OpenAI(temperature=0)
```

- **OpenAI()** initializes the language model (for example, GPT-3.5 or GPT-4).
- **llm = OpenAI(temperature=0)** sets up the agent's reasoning model.
- **temperature=0** ensures consistent and factual outputs, ideal for deterministic planning.

Code Example: Agent Execution Flow in LangChain

Step 4: Create a tool-using agent (ReAct-based)

```
agent = initialize_agent(  
    tools,  
    llm,  
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,  
    verbose=True  
)
```

- **initialize_agent(...)** creates a reasoning agent by combining tools with an LLM, enabling it to plan, act, and respond intelligently to user inputs.
- **AgentType.ZERO_SHOT_REACT_DESCRIPTION** sets the agent's strategy to ReAct, enabling dynamic tool use and reasoning without examples.
- **verbose=True** displays the agent's internal thought, action, and observation steps for debugging and transparency.

Code Example: Agent Execution Flow in LangChain

Step 5: Run the agent on a user query

```
agent.run("What's the square root of the population of India?")
```

- **agent.run("...")** prompts the agent to execute the query using its tools and reasoning.

Note:

These steps enable an LLM agent to use tools and structured reasoning for autonomous, real-world task execution.

Quick Check



During agent execution, when does the agent decide whether to call a tool or respond directly?

- A. After the memory update
- B. Immediately after parsing the input
- C. After executing the entire tool chain
- D. After receiving the user's feedback

Demo: Building a Finance Q&A Agent Using LangChain



Duration: 15 minutes

Overview: This demo illustrates how a LangChain-based agent can support product managers by interpreting business-related queries, generating executable Python code, and presenting results in plain English. It leverages Azure OpenAI alongside a Python REPL tool to bridge technical outputs with business-friendly insights.

Note:

Please download the demo document from the Reference Material section for step-by-step guidance.



Agent-to-Agent (A2A) Communication

Introduction to Agent-to-Agent Communication

It allows multiple AI agents to work together by sharing tasks and passing information through structured interactions.



Each agent focuses on one part of the task and communicates its output to others, just like a coordinated human team.

Why Use Multiple Agents

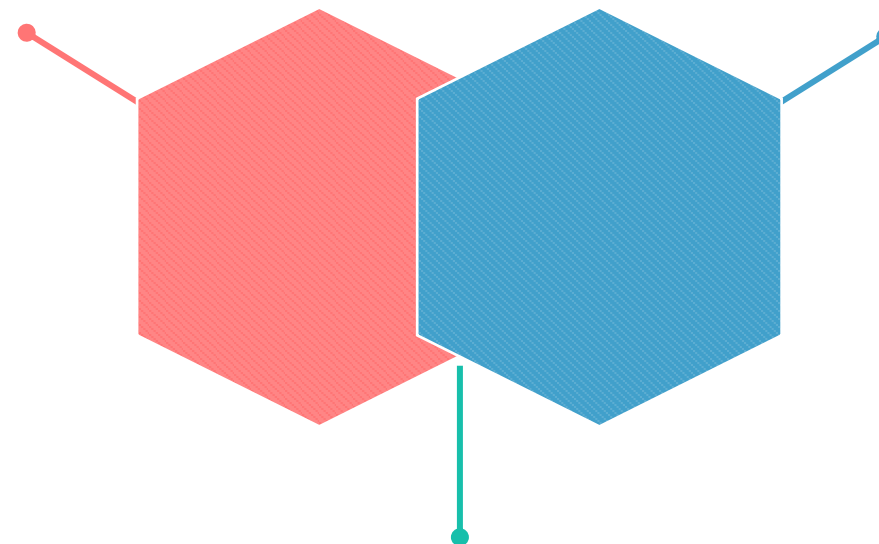
Using multiple agents offers the following key benefits in system design and execution:

Specialization

Each agent focuses on a narrow function.

Scalability

Tasks are distributed, making systems more modular and maintainable.



Resilience

Individual agent failure doesn't break the entire workflow.

Example

A travel bot uses separate agents for flights, hotels, and payments. Each is focused, replaceable, and independently resilient.

How Agents Communicate

Agents communicate via message passing, often through shared memory, queues, or message buses.

Communication includes:

Task requests

Example: Find relevant documents

State updates

Example: Task completed

Context sharing

Example: Memory, tools, constraints

Types of Agent Messages

Agents exchange different types of messages to coordinate and perform tasks efficiently:

Instruction

Triggers action in another agent



Observation

Shares results or feedback from a task



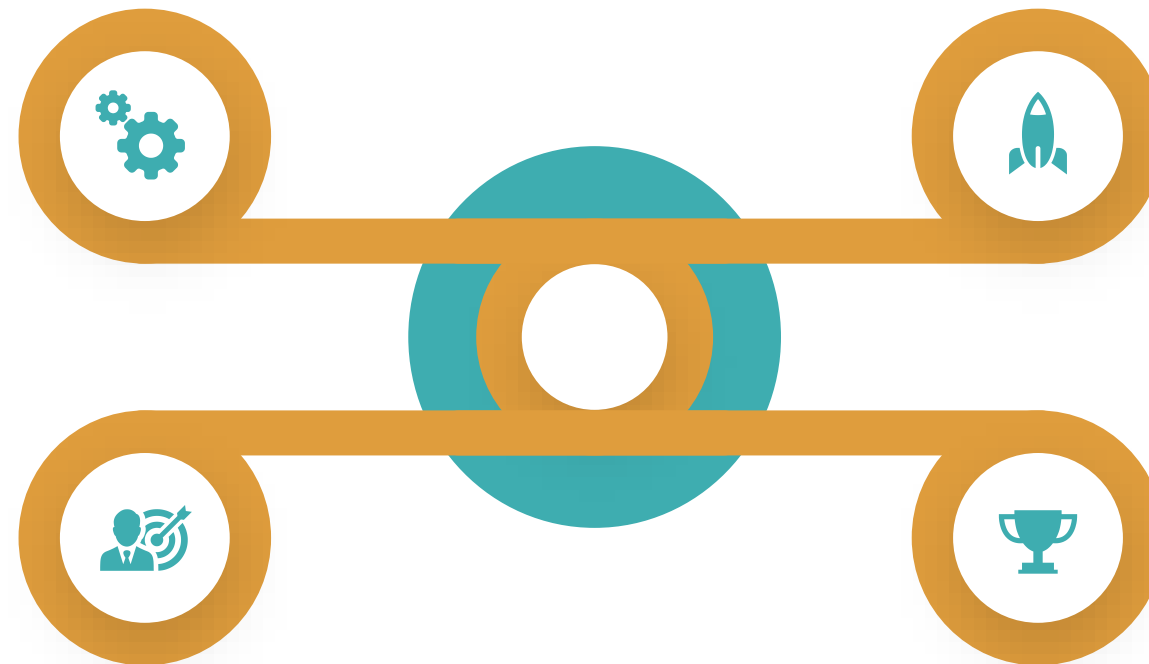
Context

Shares global memory or task constraints



Status

Shares internal state, such as confidence or availability



Designing an A2A Communication System

Step 1: Define agent goals and responsibilities

Break down the overall user task into smaller subtasks and assign each subtask to a specific agent with a focused role.

Example

Agent 1



Retrieves external data

Agent 2



Verifies accuracy

Agent 3



Generates user-facing
response

Designing an A2A Communication System

Step 2: Choose an agent communication pattern

Select how agents will coordinate and exchange information and match communication style to task urgency and complexity.

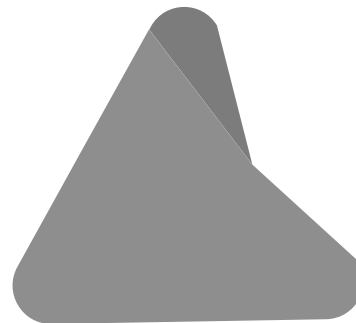
Asynchronous messaging

Agents operate independently using message queues. (Example: Payment agent)



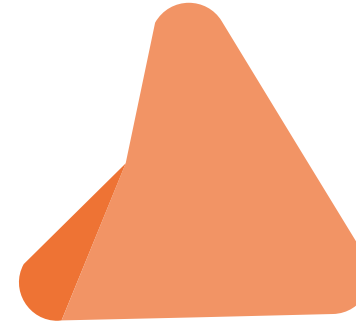
Synchronous messaging

Agents pass control in sequence, waiting for completion. (Example: Response agent)



Shared memory or blackboard

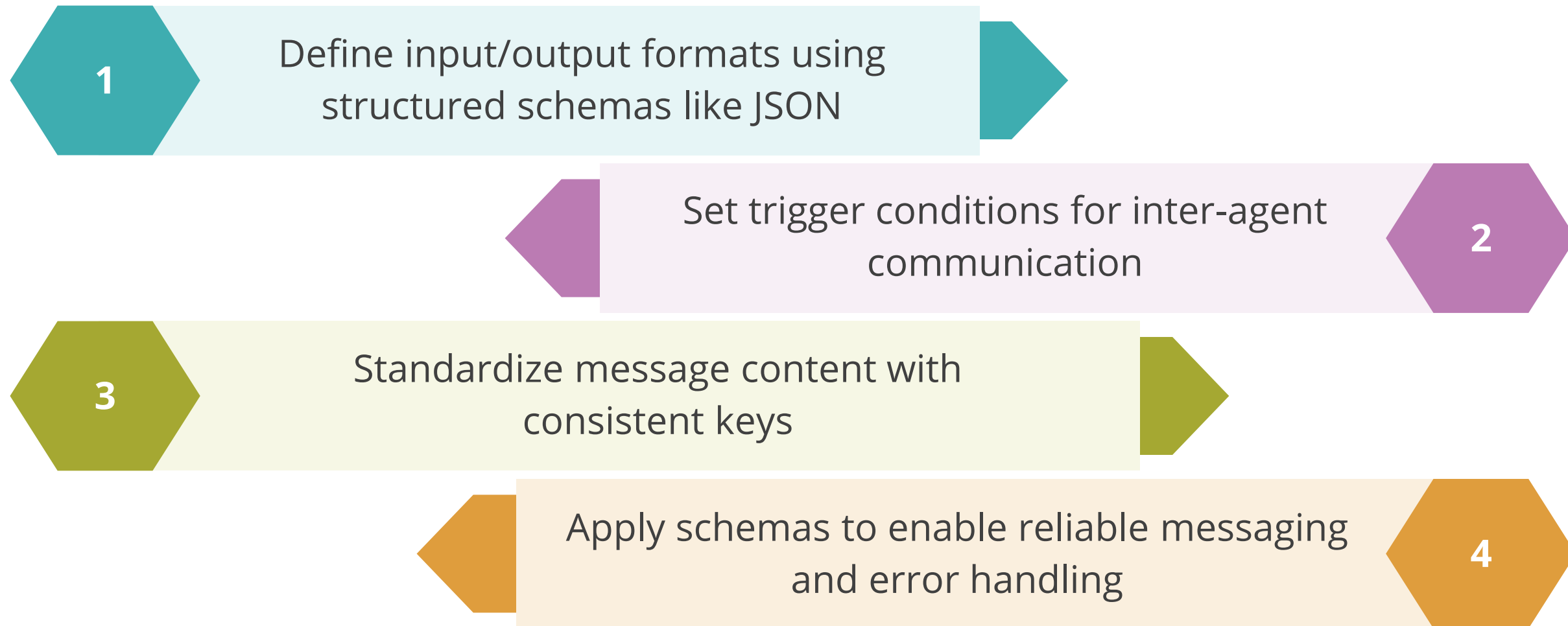
Agents read/write to a shared state or context. (Example: Research agent)



Designing an A2A Communication System

Step 3: Define message protocols and formats

Design consistent message schemas and triggers to support robust communication:



Designing an A2A Communication System

Step 4: Set the execution flow and routing logic

Define how agents make decisions, escalate tasks, and transition through workflows using:

Flowchart-based models

- Follows clear, step-by-step paths
- Handles predictable workflows
- Works well for approvals and onboarding

Rule-based methods

- Sends tasks based on set rules or keywords
- Supports flexible task flows
- Works well for helpdesk and form handling

Fallback-based agents

- Activates backup agents on failure
- Manages errors and exceptions
- Works well for support tasks

Designing an A2A Communication System

Step 5: Add traceability and control to agent systems

Add controls that ensure reliability, context retention, and accountability across agents:

Log transitions to track
agent decisions and
detect issues

Use short-term memory to retain
recent context during handoffs

Apply execution constraints like token limits,
cost thresholds, or timeouts

Real-World A2A Use Case: Sales Funnel Agents

Agents coordinate across the sales funnel to qualify leads, deliver information, and schedule demos seamlessly. The tasks performed by each agent are as follows:

Lead qualification
agent



Scores leads and
alerts product agent

Product information
agent



Fetches information
and notifies the
scheduler

Scheduling
agent



Books slots and
informs the
follow-up agent

Follow-up
agent



Sends emails and
updates the
scoring agent

Each agent plays a focused role, enabling smooth, automated sales engagement from the lead to the follow-up.

Real-World A2A Use Case: Personalized Learning Tutors

Agents collaborate to deliver personalized learning through dynamic profiling, content adaptation, and feedback. The tasks performed by each agent are as follows:

Learner profile
agent



Builds a learner
profile and flags
learning gaps

Content curation
agent



Recommends
content based
on gaps and
preferences

Assessment
agent



Creates quizzes and
sends results for
feedback

Feedback
agent



Explains mistakes
and gives
personalized tips

These agents form a closed loop that adapts instruction and support to each learner's progress in real time.

Real-World A2A Use Case: Event Planning Agents

Agents collaborate to manage events through coordinated budgeting, booking, vendor sourcing, and guest handling. The tasks performed by each agent are as follows:

Budget
agent



Tracks event costs
and flags
overspending

Venue
agent



Books venues and
shares updates
with agents

Vendor
agent



Confirms services
and aligns with the
budget and venue

Guest management
agent



Handles the invites
and updates the
vendor on guest
needs

These agents streamline end-to-end event planning by automating coordination across logistics, vendors, and guests.

Quick Check



What is the primary purpose of A2A communication in multi-agent systems?

- A. It reduces API dependencies in a single agent.
- B. It enables agents to share goals, report progress, and coordinate tasks.
- C. It helps fine-tune individual agents in isolation.
- D. It increases latency and decentralizes control.

Guided Practice



Overview

Duration: 20 minutes

You are a product manager evaluating three upcoming product features: Dark Mode, AI-based Smart Search, and Offline Support. To make a data-backed decision, you'll use LangChain agents that simulate input from internal stakeholders. Each agent scores the features based on their specific lens, usability, feasibility, or business value. You'll then run a master agent that uses these tools to return a prioritized list based on cumulative scoring.

Note:

Please download the document from the Reference Material section for step-by-step guidance.

Key Takeaways

- Planning enables agents to handle complex tasks by structuring goals and reasoning steps.
- Tools and memory help agents access external data and retain conversational context.
- RAG combines retrieval and generation to produce grounded, up-to-date outputs.
- LangChain agents orchestrate tools, memory, and logic using structured execution flows.
- Agent-to-agent communication allows collaborative task-solving through message passing.



Q&A

